

Trabajo Práctico 2: Reconstrucción 3D

Visión Artificial

Grupo 4: Sofía Bottino, Agustina Barrena, Bianca Paz

Información de contacto: sbottino@udesa.edu.ar, abarrena@udesa.edu.ar, bpaz@udesa.edu.ar

Segundo semestre 2026

Resumen

En este trabajo reconstruimos en 3D una figura plástica de un soldado sentado a partir de 31 pares de imágenes tomados con una cámara estéreo alrededor del objeto. Calibramos la cámara con la herramienta de la cátedra (error RMS de 0.67 px, línea base de 60.5 mm), rectificamos cada par y calculamos su mapa de disparidad con SGBM, que convertimos en una nube de puntos local en milímetros. Para combinar las vistas, estimamos la pose de la cámara en cada toma respecto de un tablero ChArUco con `solvePnP` (26 de los 31 pares, con un error de reproyección medio de 1.9 px) y llevamos todas las nubes al sistema del tablero. La nube fusionada, de 18.3 millones de puntos, se optimizó con Open3D mediante un filtrado espacial, el descarte de píxeles saturados, un submuestreo con voxel de 1 mm y la eliminación de outliers por radio, hasta quedar en 250.277 puntos (6.8 MB en `.ply` binario). Como opcionales, refinamos las poses con ICP y generamos una malla de Poisson. Sobre la nube final estimamos la altura del objeto en 155.8 mm, frente a 152 mm medidos con regla, un error relativo del 2.5 %.

1. Introducción y Marco Teórico

En el presente trabajo estudiamos la reconstrucción 3D de objetos a partir de imágenes, un problema central de la visión artificial con aplicaciones en robótica, inspección industrial y digitalización de objetos. Lo abordamos con el objetivo de obtener un modelo tridimensional de un objeto real, en forma de nube de puntos, y estimar sus dimensiones en unidades métricas. Para ello utilizamos visión estéreo: dos cámaras calibradas que observan la escena desde posiciones apenas distintas, y el desplazamiento aparente de cada punto entre ambas imágenes permite recuperar su profundidad por triangulación. Como cada par estéreo solo captura la cara del objeto visible desde ese punto de vista, combinamos varias vistas tomadas alrededor del objeto, expresándolas en un sistema de referencia común definido por un patrón conocido presente en la escena.

1.1. Modelo de cámara pinhole

El modelo pinhole describe la formación de la imagen como una proyección central: cada punto de la escena se proyecta en el plano imagen a través de un único punto, el centro óptico de la cámara. Siguiendo la convención de OpenCV (eje Z hacia adelante, X hacia la derecha e Y hacia abajo), un punto 3D $\mathbf{X}$ en coordenadas del mundo se proyecta en el píxel $\mathbf{x}$, en coordenadas homogéneas, según

$$\mathbf{x} = P\,\mathbf{X}, \qquad P = K\,[\,R \mid t\,], \qquad K = \begin{bmatrix} f_x & s & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}. \tag{1}$$

La matriz K contiene los *parámetros intrínsecos*: las distancias focales f_x, f_y expresadas en píxeles (pueden diferir si los píxeles no son cuadrados), el punto principal (c_x, c_y), donde el eje óptico corta al plano imagen, y el *skew* s, habitualmente nulo. La rotación R y la traslación t son los *parámetros extrínsecos*, que describen la pose de la cámara respecto del mundo. Como P es de 3×4, la proyección no es invertible: todos los puntos de un mismo rayo caen en el mismo píxel, y con una sola imagen se pierde la profundidad.

Las lentes reales se apartan de este modelo lineal. La *distorsión radial* (de tipo barril o almohadón) se modela con un polinomio en la distancia al centro, con coeficientes k_1, k_2, k_3, y la *distorsión tangencial*, causada por la falta de paralelismo entre lente y sensor, con p_1, p_2. Como la función de distorsión no tiene una inversa cerrada, su corrección se precomputa como un mapa de corrimientos de coordenadas.

1.2. Calibración de la cámara estéreo

Calibrar consiste en estimar los parámetros intrínsecos y de distorsión de cada cámara, sin los cuales no es posible relacionar un píxel con el rayo 3D que lo generó ni corregir las deformaciones de la lente. Se utiliza un patrón de geometría conocida, como un tablero de ajedrez: sus esquinas tienen coordenadas 3D conocidas en el sistema del tablero (sobre el plano $Z = 0$, separadas por el lado del cuadrado), y al detectarlas en varias imágenes tomadas desde distintas orientaciones se obtienen correspondencias 2D–3D. El método de Zhang usa estas correspondencias para estimar K, los coeficientes de distorsión y la pose del tablero en cada imagen, y luego refina todos los parámetros minimizando el *error de reproyección*:

$$e = \sum_i \sum_j \left\| \mathbf{x}_{ij} - \pi(K, \mathrm{dist}, R_i, t_i, \mathbf{X}_j) \right\|^2, \tag{2}$$

es decir, la distancia en píxeles entre las esquinas detectadas $\mathbf{x}_{ij}$ y la proyección π de sus coordenadas 3D con los parámetros estimados. Su valor medio cuadrático (RMS) es la medida usual de la calidad de la calibración.

En la calibración estéreo, ambas cámaras observan el tablero simultáneamente, lo que permite estimar además los *parámetros extrínsecos relativos*: la rotación R y la traslación t que llevan un punto del sistema de la cámara izquierda al de la derecha, $\mathbf{X}_{der} = R\,\mathbf{X}_{izq} + t$. La norma $b = \|t\|$ es la línea base, y a partir de R, t y las matrices intrínsecas se obtienen las matrices esencial y fundamental que se usan en la rectificación.

1.3. Geometría epipolar y rectificación estéreo

Dos cámaras observando la escena definen, para cada punto 3D, un *plano epipolar* que contiene al punto y a ambos centros ópticos; la recta que une los centros es la *línea base* (de longitud b) y su intersección con cada plano imagen es el *epipolo*. El plano epipolar corta a la imagen derecha en la *línea epipolar* l', sobre la cual debe estar el correspondiente x' de un punto x de la imagen izquierda. Esta restricción se expresa como

$$x'^{\top} F\, x = 0, \qquad F = K'^{-\top} E\, K^{-1}, \qquad E = [t]_{\times} R, \tag{3}$$

donde E y F son las matrices esencial y fundamental, y reduce la búsqueda de correspondencias de dos dimensiones a una. Sin embargo, en un par general las líneas epipolares están inclinadas (todas pasan por el epipolo) y habría que calcular una por cada píxel.

La *rectificación estéreo* resuelve esto reproyectando ambos planos imagen a un plano común paralelo a la línea base. Como rotar una cámara alrededor de su centro óptico no requiere conocer la profundidad de la escena, esto se logra con dos homografías, una por imagen. El algoritmo de Bouguet elige una rotación R_{rect} cuyo nuevo eje X es la dirección de la línea base, lo que lleva el epipolo al infinito en X: las líneas epipolares quedan horizontales y en la misma fila en ambas imágenes ($y = y'$). En la práctica, la corrección de distorsión y la rectificación se precomputan como

mapas de corrimiento de coordenadas (*lookup tables*) que se aplican a cada imagen.

1.4. Correspondencia estéreo y mapa de disparidad

Con las imágenes rectificadas, encontrar el correspondiente de un píxel se reduce a deslizar un parche a lo largo de su fila en la imagen derecha y elegir el desplazamiento en el que ambos parches son más parecidos según una medida de costo, como la suma de diferencias absolutas (*block matching*). Ese desplazamiento horizontal es la *disparidad*, $d = x - x'$, y la imagen que la asigna a cada píxel es el *mapa de disparidad*. El tamaño del parche implica un compromiso: uno chico conserva detalle pero es ruidoso, y uno grande suaviza el mapa pero falla en los bordes de los objetos. El método falla en zonas sin textura, donde todos los parches son parecidos, y también con patrones repetitivos, superficies especulares y oclusiones. Para obtener mapas más coherentes, el método *Semi-Global Block Matching* (SGBM) [1] agrega al costo de cada píxel una penalización cuando sus vecinos tienen disparidades distintas, lo que favorece superficies suaves sin impedir los saltos en los bordes de los objetos.

1.5. Profundidad

Para dos cámaras paralelas con los mismos parámetros intrínsecos, separadas por la línea base b, la semejanza de triángulos da $x = fX/Z$ y $x' = f(X - b)/Z$, de donde

$$d = x - x' = \frac{b\,f}{Z} \qquad \Longrightarrow \qquad Z = \frac{b\,f}{d}. \tag{4}$$

La disparidad es inversamente proporcional a la profundidad: los objetos cercanos se desplazan más que los lejanos, y $d \to 0$ corresponde a $Z \to \infty$. Como d se mide en píxeles, la profundidad queda discretizada, y el error crece con el cuadrado de la distancia:

$$\Delta Z \approx \frac{Z^2}{b\,f}\,\Delta d. \tag{5}$$

Conocida Z, las otras coordenadas se obtienen invirtiendo la proyección, $X = (x - c_x)\,Z/f$ e $Y = (y - c_y)\,Z/f$, lo que transforma el mapa de disparidad en una *nube de puntos* 3D. Como la línea base b se conoce en unidades métricas por la calibración, la reconstrucción queda en escala real, a diferencia del caso monocular, donde la escena solo puede recuperarse salvo un factor de escala.

1.6. Estimación de pose y sistema de referencia común

Cada nube de puntos queda expresada en el sistema de la cámara que la generó. Para combinarlas hace falta conocer la pose de la cámara en cada toma respecto de un sistema fijo, el *mundo*. Si en la escena hay un objeto de geometría conocida, sus puntos 3D (en el sistema del objeto) y sus proyecciones 2D en la imagen permiten estimar esa pose resolviendo el problema *Perspective-n-Point* (PnP): conocida K, se buscan la rotación R y la traslación t que minimizan el error de reproyección de esos puntos. Usamos como referencia un tablero ChArUco, que combina un tablero de ajedrez con marcadores ArUco identificables: cada esquina detectada tiene un identificador único, por lo que el tablero puede usarse aunque esté parcialmente ocluido por el objeto.

La rotación se obtiene como un *vector de Rodrigues* $\mathbf{r}$, una representación compacta cuya dirección $\mathbf{r}/\|\mathbf{r}\|$ es el eje de giro y cuya norma $\|\mathbf{r}\|$ es el ángulo de rotación; a partir de él se reconstruye la matriz R de 3×3. La pose (R, t) lleva un punto del sistema del mundo al de la cámara, $\mathbf{X}_{cam} = R\,\mathbf{X}_{mundo} + t$, por lo que cada nube local se lleva al sistema del mundo invirtiendo esa transformación:

$$\mathbf{X}_{mundo} = R^{\top}\,(\mathbf{X}_{cam} - t)\,. \tag{6}$$

2. Metodología

El pipeline implementado consiste en calibrar la cámara estéreo (una única vez) y, para cada par, rectificar, calcular la disparidad con SGBM, obtener la nube local y estimar la pose de la cámara respecto del tablero ChArUco para llevarla al marco global. Luego fusionamos y optimizamos las nubes con Open3D y medimos el objeto sobre la nube final.

2.1. Sistema de captura y datos

Utilizamos la cámara estéreo ELP-USB3D1080P02-H120 provista por la cátedra, que entrega una única imagen de 3840×1080 píxeles con ambas vistas lado a lado, que separamos en dos imágenes de 1920×1080. Como objeto elegimos una figura plástica de aproximadamente 15 cm de alto, apoyada sobre un tablero ChArUco de 5×7 cuadros (cuadros de 52.6 mm, marcadores de 31.3 mm, diccionario `DICT_6X6_250`) que define el marco de referencia global. Capturamos 31 pares moviendo la cámara alrededor del objeto, con el objeto y el tablero inmóviles, y un conjunto independiente de 28 pares para la calibración.

2.2. Calibración estéreo

La calibración se realizó en el aula con la herramienta `calib-stereo` de la cátedra, sobre un tablero de ajedrez de 10×7 cuadros (9×6 esquinas internas) de 24.2 mm de lado. La herramienta muestra en vivo la detección del tablero en ambas cámaras y permite capturar los pares en los que se detecta en las dos vistas; tomamos 28 pares variando la distancia, la inclinación y la posición del tablero en la imagen, de modo de cubrir también los bordes, donde la distorsión de la lente gran angular es mayor.

Con las correspondencias entre las esquinas detectadas y sus coordenadas en el plano del tablero,

`stereoCalibrate` estima conjuntamente las matrices intrínsecas K_{izq}, K_{der}, los cinco coeficientes de distorsión de cada cámara y la pose relativa (R, t), minimizando el error de reproyección.

2.3. Rectificación

A partir de la calibración, `stereoRectify` calcula las rotaciones R_1, R_2, las matrices de proyección rectificadas P_1, P_2 y la matriz de reproyección Q; `initUndistortRectifyMap` genera con ellas un mapa por cámara que se calcula una sola vez y se aplica a cada par con `remap`. La herramienta de la cátedra rectifica con $\alpha = 0$, que recorta la imagen hasta eliminar todo píxel inválido: con nuestra lente esto da una focal rectificada de 2704 px (frente a 598 px originales), un zoom de $\times$4,5 que deja al objeto fuera de la imagen. Por eso recalculamos la rectificación con $\alpha = -1$, que da $f = 595$ px y conserva toda la escena, con un borde curvo sin información que identificamos rectificando una imagen blanca y descartamos en las etapas siguientes.

2.4. Mapa de disparidad

Calculamos la disparidad con el método SGBM de OpenCV. El rango de búsqueda se fijó en 256 disparidades a partir de la ecuación (4), para poder medir también las tomas en las que el tablero queda muy cerca de la cámara; como consecuencia, la franja izquierda de la imagen de ese mismo ancho queda sin disparidad. Mientras que el tamaño de bloque, 9, se eligió de forma experimental: bloques más chicos dejaban huecos en la figura y más grandes degradaban los bordes y las superficies inclinadas. Para el resto de los parámetros usamos los valores recomendados por OpenCV.

2.5. Nube de puntos local

Cada mapa se convierte en puntos 3D con `reprojectImageTo3D` y la matriz Q, que aplica la ecuación (4) a todos los píxeles; las coordenadas quedan en milímetros, en el sistema de la cámara izquierda rectificada. Conservamos los píxeles con disparidad válida, fuera del borde de rectificación y con $150 < Z < 1000$ mm: el límite inferior es la profundidad mínima medible y el superior descarta el fondo, donde el error crece con Z^2 (ecuación (5)). Cada punto toma el color de su píxel. Procesamos así los 31 pares; las 26 nubes con pose válida (apartado 2.6) son las que se fusionan.

2.6. Pose de la cámara y marco global

El sistema del mundo es el del tablero ChArUco: origen en una esquina, X e Y sobre su plano y Z perpendicular, hacia adentro del tablero (por eso el objeto queda con $Z < 0$). Lo detectamos con `CharucoDetector`, que ubica los marcadores y a partir de ellos las 24 esquinas internas con su identificador. Como el objeto tapa parte del tablero, bajamos `minMarkers` de 2 a 1 (con el valor por defecto solo 10 de los 31 pares tenían pose).

La detección se hace sobre la imagen izquierda *rectificada*, porque las nubes locales están en ese sistema; por eso usamos $K = P_1[:, :3]$ y distorsión nula. `matchImagePoints` arma los pares 2D–3D (coordenadas de cada esquina en el tablero, en mm con $Z = 0$, y su posición en la imagen), y `solvePnP` estima con ellos el vector de Rodrigues $\mathbf{r}$ y la traslación t (apartado 1.6), que `Rodrigues` convierte en R. Exigimos al menos 6 esquinas, porque con pocos puntos coplanares la pose puede salir muy mal sin ningún aviso, y controlamos el resultado con el error de reproyección. Para llevar cada nube al mundo invertimos la pose (ecuación (6)) como matriz homogénea, el formato de Open3D:

$$T_{\text{cam}\to\text{mundo}} = \begin{bmatrix} R^\top & -R^\top t \\ \mathbf{0}^\top & 1 \end{bmatrix}, \tag{7}$$

cuya última columna es la posición de la cámara en el mundo.

2.7. Fusión y optimización de la nube de puntos

Después de la etapa anterior, las nubes locales de los 26 pares con pose válida ya están expresadas en el marco del tablero ChArUco, en milímetros. Por eso fusionarlas consiste solo en concatenar sus puntos (y colores) en un único objeto `PointCloud` de Open3D. La nube resultante contiene toda la escena (muñeco, tablero, mesa y fondo) y tiene muchos puntos redundantes, porque cada zona del objeto aparece en varias vistas. Para optimizarla aplicamos los pasos que siguen, en este orden, y en cada uno registramos cuántos puntos quedan.

1. Filtrado espacial. Construimos una caja virtual (`OrientedBoundingBox`) alineada con los ejes del tablero que encierra al muñeco. Con `get_point_indices_within_bounding_box` y `select_by_index` descartamos todos los puntos que quedan afuera. Los límites ($X \in [55, 235]$ mm, $Y \in [25, 265]$ mm y altura entre 8 y 180 mm) salieron de medir sobre la nube fusionada dónde está el objeto, con unos 20 mm de margen. En Y dejamos más margen para no cortar la punta del rifle. El límite inferior es 8 mm y no 0: cada pose tiene algunos milímetros de error, así que el plano del tablero no cae exactamente en $Z = 0$ y se convierte en una capa de ruido que conviene eliminar.

2. Descarte de píxeles saturados. En la nube recortada apareció una lámina plana en el medio del casco. Viene de las tomas bajas y traseras, donde detrás de la cabeza se ve una pared blanca sobreexpuesta.

Esa zona no tiene textura, así que SGBM no encuentra correspondencias, propaga la disparidad del borde del casco y reconstruye la pared como un plano a la misma profundidad que la cabeza. El filtro de outliers no la elimina porque es densa. Como cada punto conserva el color de su píxel de origen, descartamos los puntos cuyos tres canales superan 235/255 (píxeles "quemados"). El muñeco es blanco, pero casi no tiene píxeles saturados gracias a sus sombras.

3. Submuestreo. `voxel_down_sample` divide el espacio en cubos de lado `voxel_size` y reemplaza los puntos de cada cubo por su promedio (en posición y color). Así se eliminan las repeticiones entre vistas y, al promediar, se suaviza parte del ruido. Para elegir el tamaño usamos la resolución del sistema. A la distancia de trabajo ($Z \approx 246$ mm, $f \approx 595$ px, $B \approx 60{,}5$ mm), un píxel cubre sobre el objeto

$$\Delta X \approx \frac{Z}{f} \approx 0{,}4 \text{ mm}, \tag{8}$$

y un error de un píxel en la disparidad produce un error de profundidad de

$$\Delta Z \approx \frac{Z^2}{f\,B} \approx 1{,}7 \text{ mm}. \tag{9}$$

Un voxel de 1–2 mm queda entre esas dos escalas: agrupa puntos repetidos y absorbe el ruido de profundidad sin borrar detalle que la cámara realmente puede resolver. Probamos 0.5, 1 y 2 mm. Con 0.5 mm casi solo se agrupan los puntos repetidos entre vistas. Con 2 mm aparecen huecos y se pierde la forma del casco. Elegimos **1 mm**, que conserva la geometría con aproximadamente un tercio de los puntos que deja 0.5 mm.

4. Eliminación de outliers. Después del submuestreo quedan puntos flotantes (manchas y rayas sueltas) producidos por errores de matching en distintas vistas. Comparamos dos filtros:

- `remove_statistical_outlier(nb_neighbors=20, std_ratio=2.0)`: descarta los puntos cuya distancia media a sus vecinos supera en más de 2 desvíos al promedio.
- `remove_radius_outlier(nb_points, radius)`: descarta los puntos que tienen menos de `nb_points` vecinos dentro de una esfera de radio `radius`.

Elegimos el radio según la densidad de la nube. Con voxel de 1 mm, una esfera de 5 mm centrada en la superficie del objeto contiene más de 100 puntos (es una superficie 2D con ∼2 mm de espesor). Una raya, que es esencialmente 1D, contiene unos 10, y una mancha aislada todavía menos. El filtro estadístico casi no limpió la nube: los puntos de una raya están cerca entre sí a lo largo de la línea, así que su distancia media a los vecinos se parece a la de la superficie. El filtro por radio con 3 mm todavía dejaba manchas. Con **100 puntos en 5 mm** la nube queda limpia y se conservan los detalles finos, como el rifle.

Exportación. La nube optimizada se exporta con `o3d.io.write_point_cloud` en formato `.ply` binario (`write_ascii=False`) y se verificó visualmente en MeshLab.

Opcional: refinamiento de poses con ICP. Las poses de `solvePnP` tienen un error de reproyección de 1–3 px, así que las vistas no coinciden exactamente y la superficie queda "engrosada". Para corregirlo aplicamos ICP (*Iterative Closest Point*). El algoritmo busca, para cada punto, el más cercano en la otra nube, estima la transformación rígida (R, t) que minimiza la distancia entre esos pares, la aplica y repite hasta converger. Alineamos cada vista contra la fusión de las otras 25: si la alineáramos contra la nube final, que ya la incluye, ICP encontraría su propia copia y no se movería. Como las poses del ChArUco ya dejan las nubes casi alineadas, partimos de la identidad con un umbral de correspondencia de 3 mm. Usamos la variante *point-to-plane*, que minimiza la distancia de cada punto al plano tangente de su correspondiente y converge mejor en superficies suaves. Antes de alinear, a cada vista se le aplican los mismos filtros (caja, saturados, voxel) para que el registro se haga solo sobre el muñeco. Después repetimos todo el pipeline de optimización con las nubes corregidas.

Opcional: normales y mallado de Poisson. Estimamos las normales con `estimate_normals`, que ajusta un plano a los vecinos de cada punto (radio de 3 mm, como máximo 30 vecinos) y toma su perpendicular. Después las orientamos de forma coherente entre vecinos con `orient_normals_consistent_tangent_plane`.
Con las normales generamos una malla con `create_from_point_cloud_poisson`, que busca una superficie suave cuyas normales coincidan con las de los puntos. Probamos `depth` = 7, 8 y 9 y elegimos 8: con 9 el detalle casi no mejora y el archivo pasa de 20 a 75 MB. Como Poisson siempre devuelve una superficie cerrada e "inventa" piel donde no hay datos, eliminamos el 10 % de los vértices con menor densidad de puntos de apoyo.

2.8. Medición métrica

El marco del mundo es el tablero ChArUco. El muñeco está apoyado sobre él ($Z = 0$) y, por la convención de OpenCV, su altura crece en la dirección $-Z$. Por lo tanto, la altura del objeto es la distancia del tablero a su punto más alto:

$$h = \max_i \left(-Z_i\right). \tag{10}$$

Un único punto extremo puede ser ruido, así que además reportamos el percentil 99.9 de $-Z$ como estimación robusta. También verificamos que los puntos

más altos estén agrupados sobre la cabeza y no formen una mancha aislada: calculamos la dispersión en XY de los 200 puntos más altos. Comparamos el resultado con la altura del muñeco medida con regla.

Esta medición da directamente un valor en milímetros. Con una sola cámara, la reconstrucción solo queda definida a menos de un factor de escala: un objeto el doble de grande y el doble de lejos produce la misma imagen. En nuestro pipeline, en cambio, la escala la fijan dos magnitudes conocidas en milímetros. La primera es la **baseline** $B = \|T\| \approx 60{,}5$ mm, que surge de la calibración estéreo con un tablero de cuadros de 24.2 mm. Con ella la profundidad $Z = fB/d$ queda en milímetros. La segunda es el **tablero ChArUco** (cuadros de 52.6 mm), cuyas coordenadas 3D en mm se usan en `solvePnP`. Así la traslación de cada pose, y con ella el marco global, también queda en milímetros. Como las dos referencias son métricas, la nube fusionada y las distancias medidas sobre ella están en milímetros, sin ningún reescalado posterior.

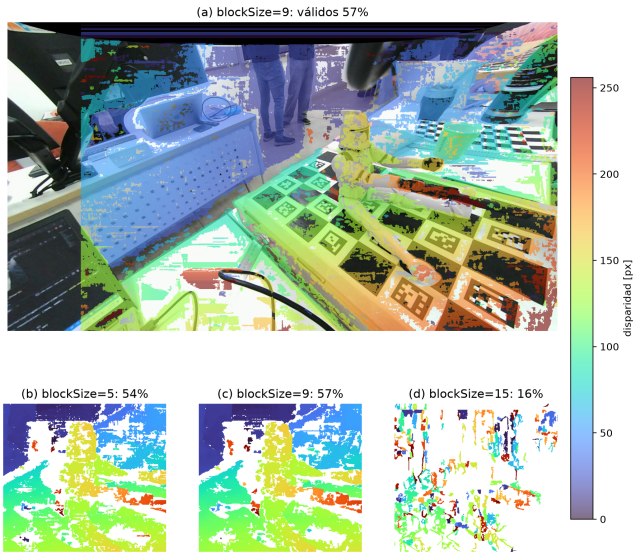


Figura 2: Disparidad del par 0 (SGBM, bloque de 9), superpuesta a la imagen izquierda rectificada. En los píxeles sin color no se encontró una disparidad válida.

3. Resultados y Discusión

3.1. Calibración

La cuadro 1 resume la calibración. Ambas cámaras tienen parámetros casi idénticos, la rotación relativa es de solo 0,5° y la traslación es casi horizontal, con $b = 60{,}5$ mm, consistente con la línea base nominal de la cámara (60 mm). El error de reproyección RMS es 0.67 px. En el par rectificado (figura 1) las líneas horizontales atraviesan los mismos detalles en ambas imágenes, y se ve que los objetos cercanos se desplazan más que el fondo.

Tabla 1: Resultados de la calibración estéreo.

	Izquierda	Derecha
f_x, f_y [px]	597.7, 594.5	596.8, 595.5
c_x, c_y [px]	942.9, 521.3	938.4, 519.6
t [mm]	$(-60{,}50,\ -0{,}36,\ 1{,}06)$	
Rotación relativa	0,5°	
RMS reproyección	0.67 px	

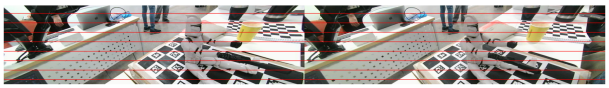


Figura 1: Franja central del par 0 rectificado ($\alpha = -1$) con líneas horizontales.

3.2. Disparidad

La figura 2 muestra el mapa del par 0, en el que el 57 % de los píxeles tiene disparidad válida. Las superficies aparecen como degradados coherentes, y la disparidad se concentra en las zonas con textura (bordes del tablero, marcadores, detalles negros de la figura), mientras que las superficies lisas quedan sin valor, porque ahí el costo de correspondencia es ambiguo y el filtro de unicidad descarta el píxel. Como verificación, en el torso la disparidad mediana es 146.3 px, que por la ecuación (4) da $Z = 246{,}0$ mm, el mismo valor que devuelve `reprojectImageTo3D`.

Sobre los 31 pares, las tomas altas (0–18) tienen en promedio un 51 % de píxeles válidos y las bajas (19–30) un 42 %: con la cámara casi a ras del tablero, este queda muy cerca y visto en un ángulo rasante. Otras limitaciones son la franja izquierda de 256 columnas sin disparidad, impuesta por el rango de búsqueda, y el borde descartado por la rectificación con $\alpha = -1$.

3.3. Pose y marco global

De los 31 pares, 26 tienen pose, con entre 6 y 20 esquinas detectadas y un error de reproyección medio de 0.8 a 3.1 px (1.9 px en promedio). Descartamos los pares 11, 15, 20, 23 y 29, en los que el objeto tapa el tablero o la toma es demasiado rasante. Con OpenCV 4.13 se pierden además los pares 25 y 28.

La figura 3 muestra las 26 cámaras junto a la nube final, y sus posiciones coinciden con la captura: rodean casi por completo al objeto (unos 320°), a entre 17 y 36 cm de su centro, en dos vueltas, una alta (pares 0–18, a 12–31 cm sobre el tablero) y una baja (pares 19–30, a 5–16 cm). Todos los ejes ópticos apuntan al objeto y las nubes de las distintas vistas se superponen sin duplicados visibles, lo que valida las poses y la inversión de la ecuación (6).

Las poses no son exactas: con un píxel de unos 0.4 mm sobre el objeto, 1–3 px de error se traducen en desalineaciones de alrededor de 1 mm entre vistas, coherentes con el RMSE de 1.03 mm previo al ICP. El error tampoco es uniforme: en el par 0 el medio es 2.3 px, pero una esquina llega a 13.3 px. Las esquinas aceptadas con un solo marcador o parcialmente tapa-

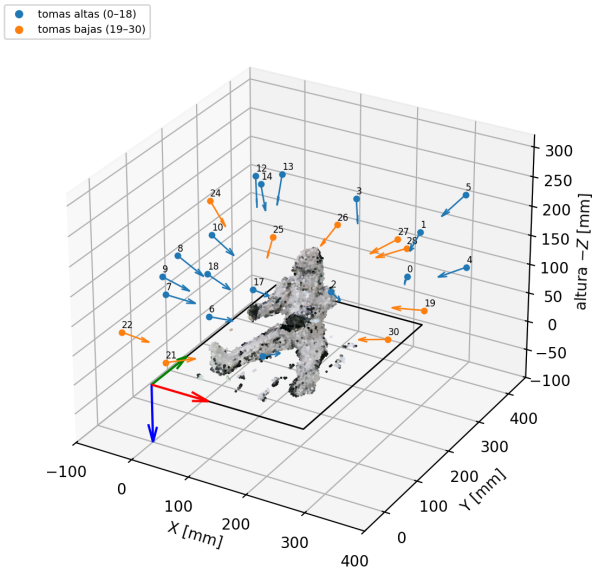


Figura 3: Posición de las 26 cámaras con pose (punto) y dirección de su eje óptico (flecha) en el sistema del tablero (rectángulo negro), junto a la nube final. Se grafica la altura $-Z$.

das se ubican peor, y `solvePnP` las pesa igual que a las demás. Se podría usar `solvePnPRansac` para descartarlas, o triangular las esquinas con las dos cámaras.

3.4. Nube global y optimización

Al fusionar las nubes de los 26 pares con pose se obtuvo una nube de 18.333.114 puntos que contiene toda la escena. La Tabla 2 muestra cuántos puntos quedan después de cada paso de optimización.

Tabla 2: Puntos sobrevivientes en cada etapa de la optimización. "Del paso" indica el porcentaje que se conserva respecto de la etapa anterior; "Del total", respecto de la nube fusionada original.

Etapa	Puntos	Del paso	Del total
Fusión	18.333.114	–	100 %
Filtrado espacial	1.908.379	10.4 %	10.4 %
Píxeles saturados	1.836.268	96.2 %	10.0 %
Submuestreo	299.524	16.3 %	1.6 %
Outliers	250.277	83.6 %	1.4 %

Filtrado espacial. Es el paso que más puntos elimina: casi el 90 % de la nube fusionada corresponde a la mesa, el tablero y el fondo. Funciona bien, pero tiene una limitación: los límites de la caja se eligieron a mano mirando la nube, así que el procedimiento no se puede aplicar directamente a otro objeto o escena. Una mejora sería automatizarlo: segmentar el plano del tablero con RANSAC (`segment_plane`) y aislar el objeto con clustering (DBSCAN).

Píxeles saturados. Con este paso se eliminan 72.111 puntos (3.8 %), concentrados en la lámina reconstruida a partir de la pared sobreexpuesta y en algunas rayas del fondo, sin afectar al muñeco (Figura 4). Este fue un problema que encontramos al inspeccionar la nube recortada. SGBM no puede hacer matching en zonas sin textura y propaga la disparidad del borde vecino, así que genera un plano denso que ningún filtro geométrico de outliers detecta. El umbral de color lo resuelve, pero es una solución heurística: con una iluminación más fuerte, partes blancas del propio muñeco podrían saturarse y perderse. Sería mejor evitar el problema en la captura (controlar la exposición y usar un fondo con textura) o en la disparidad, con una verificación izquierda-derecha que invalide los píxeles sin correspondencia confiable.

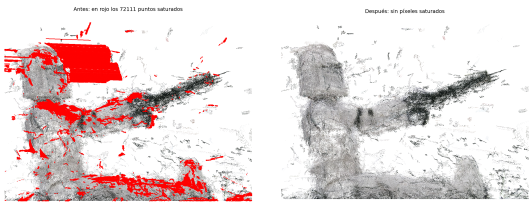


Figura 4: Nube recortada por la caja, antes (en rojo, los puntos saturados) y después de descartarlos. Vista de costado (plano Y–altura).

Submuestreo. El voxel de 1 mm reduce la nube a un 16 %. Esto confirma que la mayor parte de los puntos eran repeticiones de la misma superficie vista desde varias cámaras. En la comparación de tamaños (Figura 5), 0.5 mm conserva 992.997 puntos (54.1 %), 1 mm conserva 299.524 (16.3 %) y 2 mm conserva 71.346 (3.9 %). Con 2 mm la superficie del casco ya presenta huecos. Con 1 mm la forma se conserva sin pérdida visible, como era de esperar, ya que ese tamaño está entre la resolución lateral (∼0.4 mm) y el ruido de profundidad (∼1.7 mm).



Figura 5: Comparación de tamaños de voxel: 0.5, 1 y 2 mm.

Eliminación de outliers. El filtro estadístico (20 vecinos, 2.0 desvíos) conservó el 97.3 % de los puntos y casi no eliminó los artefactos visibles (Figura 6). Su limitación es que los puntos de una raya flotante están agrupados de forma densa a lo largo de la línea, así que su distancia media a los vecinos se parece a la de la superficie. Los filtros por radio conservaron el 83.8 % (30 puntos en 3 mm) y el 83.6 % (100 puntos en 5 mm), y solo el segundo dejó la nube visualmente limpia. El filtro por radio elimina más puntos, pero algunos de ellos pertenecen a los bordes del objeto, donde hay menos vistas y la densidad es menor.

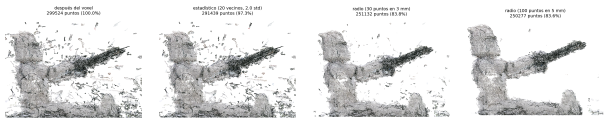


Figura 6: Comparación de filtros de outliers sobre la nube submuestreada.

Nube final. La nube final tiene 250.277 puntos, es decir, el 1.4 % de los 18.333.114 puntos de la nube fusionada original. El archivo `.ply` binario pesa 6.8 MB (Figura 7). La malla de Poisson (`depth` = 8) tiene 254.157 vértices y 505.929 triángulos (19.5 MB). Cierra bien la superficie del muñeco, aunque en las zonas con pocos datos sigue "inventando" superficie suave que no corresponde al objeto real. Además, en Open3D 0.20 Poisson fallaba al ejecutarse con varios hilos y cerraba el kernel, por eso hubo que correrlo con `n_threads=1`.

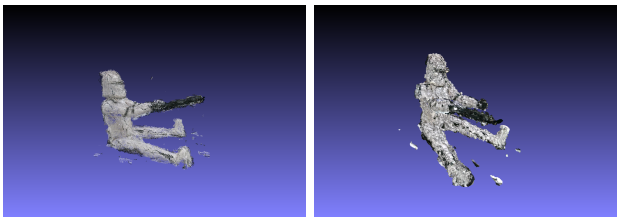


Figura 7: Nube final optimizada (izquierda) y malla de Poisson (derecha), visualizadas en MeshLab.

Refinamiento con ICP. ICP corrigió rotaciones de entre 0.2° y 2.3° por vista y redujo el RMSE medio entre cada vista y las demás de 1.03 a 0.96 mm. Con las poses corregidas, el submuestreo deja 5.7 % menos puntos (282.420 contra 299.524), porque las vistas se superponen mejor y caen en los mismos voxels. Es decir, la superficie reconstruida queda algo más delgada. Sin embargo, la mejora es chica. Las poses obtenidas con el ChArUco ya eran buenas, y la mayor parte del espesor de la superficie viene del ruido de SGBM dentro de cada vista, que un registro rígido no puede corregir. Por eso el entregable principal es la nube sin ICP. Para mejorar este aspecto habría que reducir el ruido de disparidad, por ejemplo con métodos de stereo matching basados en redes neuronales, o tomando capturas más cercanas al objeto, ya que el error de profundidad crece con Z^2.

3.5. Medición métrica

La altura se midió sobre la nube final como la distancia entre el plano del tablero ($Z = 0$) y el punto más alto, y además, como el percentil 99.9 de la altura. Los 200 puntos más altos están agrupados sobre la cabeza (dispersión de 3 y 5 mm en X e Y), así que el máximo no corresponde a un punto suelto. La altura real del muñeco, medida con regla, es de 152 mm. La Tabla 3 resume los resultados y la Figura 8 muestra la medición.

Tabla 3: Altura estimada del muñeco frente a la altura real (152 mm).

Método	h [mm]	Error [mm]	Error rel.
Máximo, sin ICP	159.6	+7.6	5.0 %
P99.9, sin ICP	155.8	+3.8	2.5 %
Máximo, con ICP	158.0	+6.0	4.0 %
P99.9, con ICP	155.3	+3.3	2.2 %

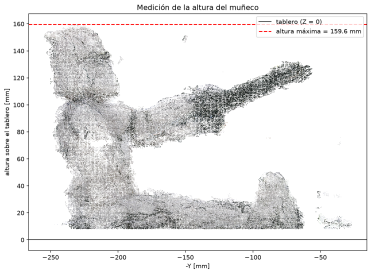


Figura 8: Medición de la altura sobre la nube final: plano del tablero ($Z = 0$) y punto más alto.

Tomamos como estimación final la del percentil 99.9 sobre la nube final optimizada (sin ICP): **155.8 mm**, con un error relativo de 2.5 %. Todas las estimaciones quedan por encima del valor real. Esto se debe a que la superficie reconstruida no es una capa fina, sino una "cáscara" de algunos milímetros de espesor, que tiene dos orígenes. El primero es el ruido de profundidad de cada vista ($\Delta Z \approx 1{,}7$ mm por píxel de disparidad). El segundo es el error de las poses: con 1–3 px de error de reproyección, las 26 nubes no coinciden exactamente. El punto más alto toma el extremo exterior de esa cáscara, por eso sobreestima más. El percentil descarta esos extremos y queda más cerca del valor real. Con ICP la cáscara se afina y las dos estimaciones mejoran, pero solo en 0.5–1.6 mm.

Hay otras dos fuentes de error. La altura se mide respecto del plano $Z = 0$ estimado por `solvePnP`, así que un error de pocos milímetros en ese plano se traslada directamente a la medida. Además, la referencia con regla también tiene un error (estimamos 1–2 mm) por la curvatura del casco. Por otro lado, solo medimos la altura: el ancho es más difícil de definir en un objeto irregular con un rifle que sobresale. Para mejorar la medición se podría ajustar un plano a la parte superior del casco o medir sobre la malla de Poisson, que ya promedia la cáscara, y repetir varias veces la medición con regla para acotar el error de referencia.

Referencias

[1] H. Hirschmüller, "Stereo Processing by Semiglobal Matching and Mutual Information," *IEEE Trans. Pattern Analysis and Machine Intelligence*, vol. 30, no. 2, 2008.